

Cola de Python: Referência Rápida para Lógica

Redes de Computadores – IFCE Campus Tauá

1. Entrada, Saída e Conversão

Para interagir com o usuário, ler do teclado e exibir dados formatados.

```
# Entrada de dados (lidos como texto/string)
texto = input("Digite algo: ")
# Conversao de tipos
inteiro = int(input("Digite um inteiro: "))
decimal = float(input("Digite um decimal: "))

# Saida com f-strings (chaves {} para variaveis)
print(f"Texto: {texto} | Inteiro: {inteiro}")
# Formatar com casas decimais (Ex: 2 casas)
print(f"Decimal formatado: {decimal:.2f}")
```

2. Operações Matemáticas

Operadores aritméticos básicos para cálculos no Python.

```
a = 10
b = 3

soma = a + b      # 13
sub = a - b       # 7
mult = a * b      # 30
div = a / b       # 3.3333... (divisao real)
div_inteira = a // b # 3 (descarta a parte decimal)
resto = a % b     # 1 (resto da divisao, útil p/ par/
                 # impar)
potencia = a ** b # 1000 (10 elevado a 3)
```

3. Manipulação de Strings

Métodos úteis para tratar textos e strings.

```
texto = " Algoritmos em Python! "

maiusculo = texto.upper()    # " ALGORITMOS EM PYTHON!
                              # "
minusculo = texto.lower()    # " algoritmos em python!
                              # "
limpo = texto.strip()        # "Algoritmos em Python!" (
                              # remove espacos)
trocado = texto.replace("Python", "C") # " Algoritmos em
                              # C! "

# Dividir texto em lista por um separador
frase = "arroz,feijao,carne"
itens = frase.split(",")     # ['arroz', 'feijao', '
                              # carne']

# Juntar lista em texto
junto = "-".join(itens)      # "arroz-feijao-carne"
```

4. Decisões e Condições (if)

Executa blocos de código com base em testes lógicos.

```
# Comparadores: ==, !=, >, <, >=, <=
# Operadores logicos: and, or, not

# --- IF SIMPLES (Verificacao unica) ---
nota = 8.5
if nota >= 7.0:
    print("Aprovado!")

# --- IF COMPOSTO (Multiplos caminhos) ---
idade = 17
tem_autorizacao = True

if idade >= 18:
    print("Acesso liberado (Maior de idade)")
elif idade >= 16 and tem_autorizacao:
    print("Acesso liberado (Com autorizacao)")
else:
    print("Acesso negado")
```

5. Repetições (Loops: for e while)

Repete um bloco de código de forma controlada.

```
# --- LOOP FOR COM RANGE ---
# range(fim) -> vai de 0 ate fim-1
for i in range(5): # 0, 1, 2, 3, 4
    print(i)

# range(inicio, fim) -> de inicio ate fim-1
for i in range(2, 6): # 2, 3, 4, 5
    print(i)

# range(inicio, fim, passo) -> com saltos
for i in range(10, 21, 2): # 10, 12, 14, 16, 18, 20
    print(i)

# --- LOOP WHILE (Enquanto) ---
contador = 1
while contador <= 5:
    print(contador)
    contador += 1 # Evita loop infinito!
```

6. Listas (Vetores de 1 Dimensão)

Armazena múltiplos valores ordenados.

```
numeros = [10, 20, 30, 40]
primeiro = numeros[0] # 10
ultimo = numeros[-1] # 40 (fim da lista)

# Inserir e Modificar
numeros[1] = 25 # [10, 25, 30, 40]
numeros.append(50) # Final: [10, 25, 30, 40, 50]
numeros.insert(1, 15) # No indice 1: [10, 15, 25, 30, 40,
50]

# Remover
numeros.pop() # Remove ultimo: [10, 15, 25, 30,
40]
numeros.pop(1) # Remove do indice 1: [10, 25, 30,
40]
numeros.remove(30) # Remove valor 30: [10, 25, 40]

# Ordenacao e Inversao
valores = [5, 2, 9, 1]
valores.sort() # Altera p/ crescente: [1, 2, 5, 9]
valores.sort(reverse=True) # Decrescente: [9, 5, 2, 1]
valores.reverse() # Inverte a ordem da lista

# Iterar com indices
for i, num in enumerate(valores):
    print(f"Indice {i} tem o valor {num}")
```

7. Matrizes (Listas de Listas)

Coleções organizadas em linhas e colunas (2 dimensões).

```
matriz = [
    [1, 2, 3], # Linha 0
    [4, 5, 6], # Linha 1
    [7, 8, 9] # Linha 2
]

# Acesso: matriz[linha][coluna]
valor = matriz[1][2] # Linha 1, Coluna 2 -> valor 6
matriz[0][0] = 99 # Altera valor na Linha 0, Coluna 0

# Percorrer linha por linha
for linha in matriz:
    for elemento in linha:
        print(elemento, end="\t")
    print() # Quebra de linha ao fim de cada linha

# Percorrer usando indices
linhas = len(matriz)
colunas = len(matriz[0])
for i in range(linhas):
    for j in range(colunas):
        print(f"matriz[{i}][{j}] = {matriz[i][j]}")
```

8. Dicionários (Chave-Valor)

Armazena dados associados a rótulos (chaves).

```
aluno = {
    "nome": "Joao Pedro",
    "idade": 20,
    "curso": "ADS"
}
print(aluno["nome"]) # "Joao Pedro"

# Adicionar/Modificar
aluno["media"] = 8.5 # Cria nova chave
aluno["idade"] = 21 # Modifica chave existente

# Remocao
del aluno["curso"]
valor = aluno.pop("idade")

# Percorrer Chaves e Valores
for chave, valor in aluno.items():
    print(f"{chave}: {valor}")
```

9. Funções Embutidas Úteis

Funções globais que facilitam cálculos comuns.

Função	Descrição	Exemplo
len(c)	Tamanho da coleção	len([4, 8, 2]) → 3
sum(c)	Soma elementos	sum([4, 8, 2]) → 14
min(c)	Menor elemento	min([4, 8, 2]) → 2
max(c)	Maior elemento	max([4, 8, 2]) → 8
sorted(c)	Cria lista ordenada	sorted([4, 8, 2]) → [2, 4, 8]
abs(n)	Valor absoluto	abs(-15) → 15
round(n,d)	Arredonda p/ d casas	round(3.1415, 2) → 3.14

10. Padrões de Algoritmos

Soluções clássicas aplicadas a listas de dados.

A. Acumulador (Soma acumulada)

Soma valores com base em critérios.

```
valores = [12, 5, 8, 21, 30, 4]
soma_pares = 0
for v in valores:
    if v % 2 == 0:
        soma_pares += v
```

B. Contador

Conta ocorrências de determinado critério.

```
notas = [6.5, 8.0, 5.5, 9.0, 4.0, 7.0]
aprovados = 0
for nota in notas:
    if nota >= 6.0:
        aprovados += 1
```

C. Maior/Menor Elemento (Manual)

Algoritmo de extremos sem usar funções embutidas.

```
idades = [25, 42, 18, 50, 31]
maior_idade = idades[0]
for idade in idades:
    if idade > maior_idade:
        maior_idade = idade
```

D. Filtragem de Listas

Gera uma nova lista com elementos filtrados.

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
impares = []
for n in numeros:
    if n % 2 != 0:
        impares.append(n)
```

11. List Comprehension

Forma curta de criar e filtrar listas em Python.

```
# --- SINTAXE BASICA ---
# [expressao for item in colecao]
quadrados = [x**2 for x in range(5)] # [0, 1, 4, 9, 16]

# --- SINTAXE COM FILTRO (IF) ---
# [expressao for item in colecao if condicao]
pares = [x for x in range(10) if x % 2 == 0] # [0, 2, 4, 6, 8]

# --- EXEMPLOS TEMATICOS (REDES) ---
# 1. Limpar espacos de strings em uma lista de IPs
hosts = [" 192.168.1.1 ", " 10.0.0.1", "172.16.0.254 "]
ips_limpos = [ip.strip() for ip in hosts]
# Resultado: ['192.168.1.1', '10.0.0.1', '172.16.0.254']

# 2. Filtrar apenas IPs locais (começam com "192.")
ips = ["192.168.1.5", "10.0.0.3", "192.168.2.10", "8.8.8.8"]
ips_locais = [ip for ip in ips if ip.startswith("192.")]
# Resultado: ['192.168.1.5', '192.168.2.10']
```